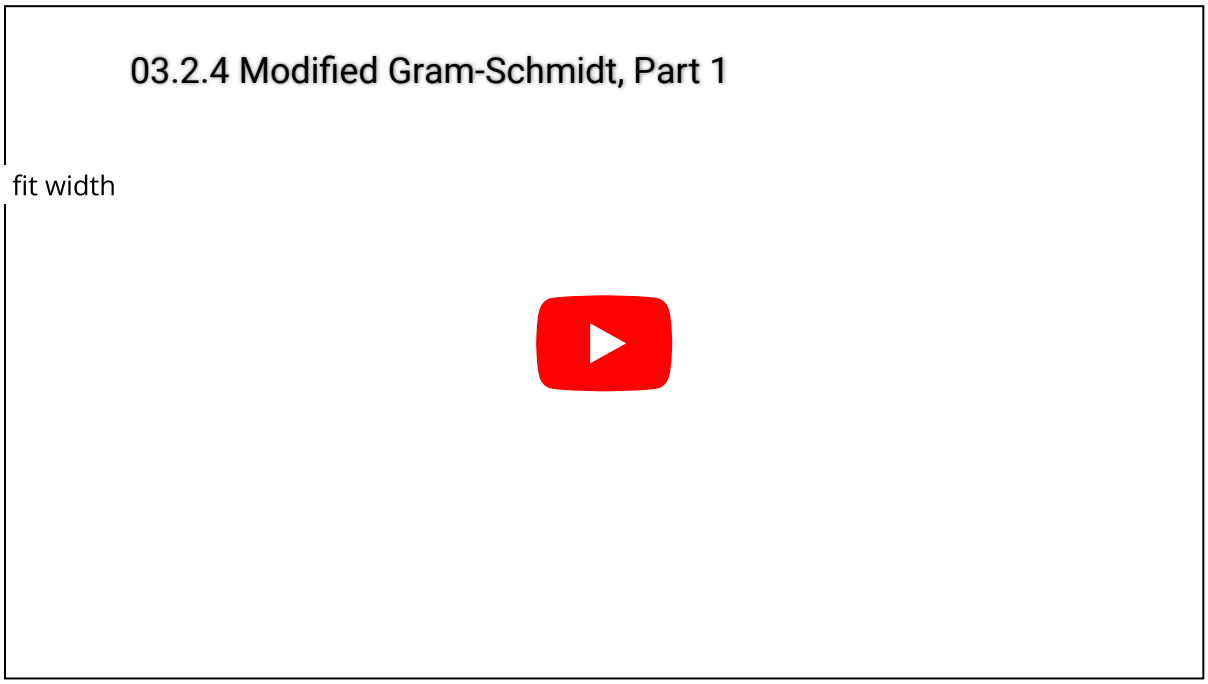
3.2.4 Modified Gram-Schmidt (MGS)



In the video, we reasoned that the following two algorithms compute the same values, except that the columns of Q overwrite the corresponding columns of A:

for $j = 0, \ldots, n-1$
  $a_j^\perp := a_j$
  for $k = 0, \ldots, j-1$
    $\rho_{k,j} := q_k^H a_j^\perp$
    $a_j^\perp := a_j^\perp - \rho_{k,j} q_k$
  end
  $\rho_{j,j} := \|a_j^\perp\|_2$
  $q_j := a_j^\perp / \rho_{j,j}$
end

(a) MGS algorithm that computes Q and R from A.

for $j = 0, \ldots, n-1$

  for $k = 0, \ldots, j-1$
    $\rho_{k,j} := a_k^H a_j$
    $a_j := a_j - \rho_{k,j} a_k$
  end
  $\rho_{j,j} := \|a_j\|_2$
  $a_j := a_j / \rho_{j,j}$
end

(b) MGS algorithm that computes Q and R from A, overwriting A with Q.

Homework 3.2.4.1. Assume that $q_0, \ldots, q_{k-1}$ are mutually orthonormal. Let $\rho_{j,k} = q_j^H y$ for $j = 0, \ldots, i-1$. Show that

$$\underbrace{q_i^H y}_{\rho_{i,k}} = q_i^H \left(y - \rho_{0,k} q_0 - \cdots - \rho_{i-1,k} q_{i-1} \right)$$

for $i = 0, \ldots, k-1$.

▼ Solution

$$\begin{array}{l} q_i^H \left(y - \rho_{0,k} q_0 - \cdots - \rho_{i-1,k} q_{i-1} \right) \\ \quad = \quad < \text{ distribute } > \\ q_i^H y - q_i^H \rho_{0,k} q_0 - \cdots - q_i^H \rho_{i-1,k} q_{i-1} \\ \quad = \quad < \rho_{0,k} \text{ is a scalar } > \\ q_i^H y - \rho_{0,k} \underbrace{q_i^H q_0}_{0} - \cdots - \underbrace{\rho_{i-1,k} q_i^H q_{i-1}}_{0} \end{array}$$

03.2.4 Modified Gram-Schmidt, Part 2

fit width

This homework illustrates how, given a vector $y \in \mathbb{C}^m$ and a matrix $Q \in \mathbb{C}^{mxk}$ the component orthogonal to the column space of Q, given by $(I - QQ^H)y$, can be computed by either of the two algorithms given in Figure 3.2.4.1. The one on the left, Proj $\perp$ $Q_{\text{CGS}}(Q, y)$ projects y onto the column space perpendicular to Q as did the Gram-Schmidt algorithm with which we started. The one on the left successfuly subtracts out the component in the direction of q_i using a vector that has been updated in previous iterations (and hence is already orthogonal to $q_0, \ldots, q_{i-1}$). The algorithm on the right is one variant of the Modified Gram-Schmidt (MGS) algorithm.

$[y^\perp, r] = \text{Proj}\perp\text{Q}_{\text{CGS}}(Q, y)$ (used by CGS)	$[y^\perp, r] = \text{Proj}\perp\text{Q}_{\text{MGS}}(Q, y)$ (used by MGS)
$y^\perp = y$ **for** $i = 0, \ldots, k-1$ $\rho_i := q_i^H y$ $y^\perp := y^\perp - \rho_i q_i$ **endfor**	$y^\perp = y$ **for** $i = 0, \ldots, k-1$ $\rho_i := q_i^H y^\perp$ $y^\perp := y^\perp - \rho_i q_i$ **endfor**

Figure 3.2.4.1. Two different ways of computing $y^\perp = (I - QQ^H)y = y - Qr$, where $r = Q^H y$. The computed $y^\perp$ is the component of y orthogonal to $\mathcal{C}(Q)$, where Q has k orthonormal columns. (Notice the y on the left versus the $y^\perp$ on the right in the computation of ρ_i.)

These insights allow us to present CGS and this variant of MGS in FLAME notation, in Figure 3.2.4.2 (left and middle).

$[A, R] := \text{GS}(A)$ (overwrites A with Q)

$$A \rightarrow \left(\begin{array}{c|c} A_L & A_R \end{array} \right), R \rightarrow \left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right)$$

A_L has 0 columns and R_{TL} is 0×0

while $n(A_L) < n(A)$

$$\left(\begin{array}{c|c} A_L & A_R \end{array} \right) \rightarrow \left(\begin{array}{c|cc} A_0 & a_1 & A_2 \end{array} \right), \left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c|cc} R_{00} & r_{01} & R_{02} \\ \hline 0 & \rho_{11} & r_{12}^T \\ 0 & 0 & R_{22} \end{array} \right)$$

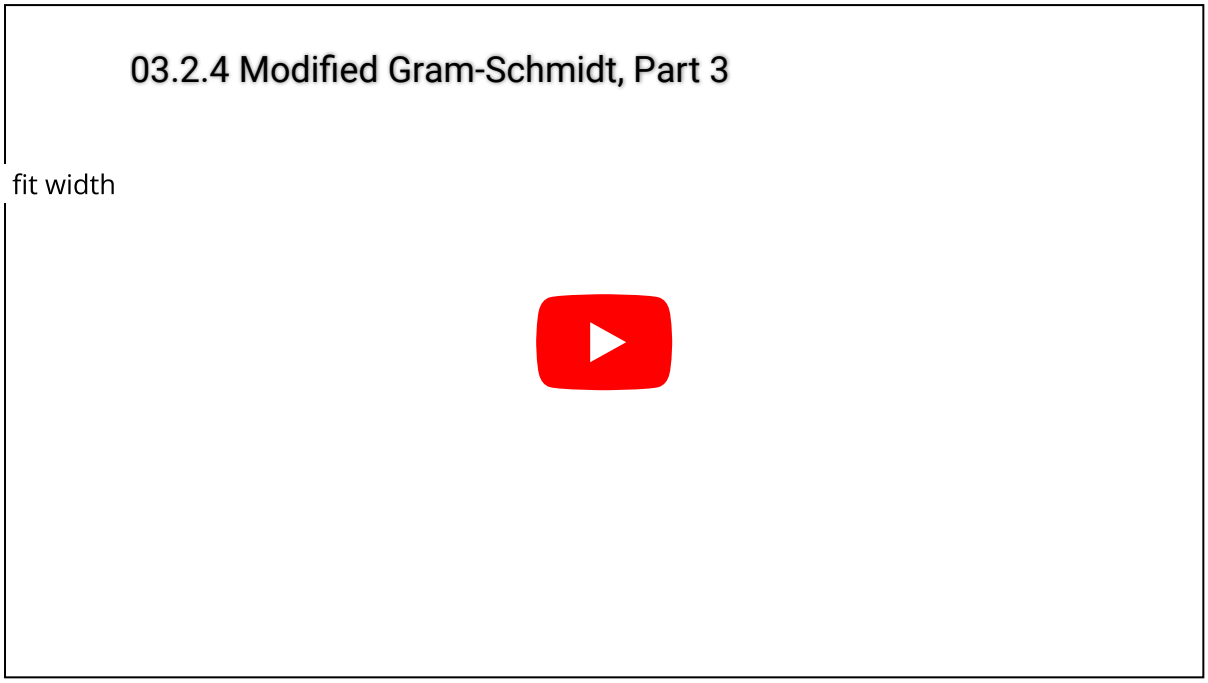
CGS	MGS	MGS (alternative)
$r_{01} := A_0^H a_1$		
$a_1 := a_1 - A_0 r_{01}$	$[a_1, r_{01}] = \text{Proj}\perp\text{toQ}_{\text{MGS}}(A_0, a_1)$	
$\rho_{11} := \|a_1\|_2$	$\rho_{11} := \|a_1\|_2$	$\rho_{11} := \|a_1\|_2$
$a_1 := a_1/\rho_{11}$	$a_1 := a_1/\rho_{11}$	$a_1 := a_1/\rho_{11}$
		$r_{12}^T := a_1^H A_2$
		$A_2 := A_2 - a_1 r_{12}^T$

$$\left(\begin{array}{c|c} A_L & A_R \end{array} \right) \leftarrow \left(\begin{array}{cc|c} A_0 & a_1 & A_2 \end{array} \right), \left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right) \leftarrow \left(\begin{array}{cc|c} R_{00} & r_{01} & R_{02} \\ 0 & \rho_{11} & r_{12}^T \\ \hline 0 & 0 & R_{22} \end{array} \right)$$

endwhile

Figure 3.2.4.2. Left: Classical Gram-Schmidt algorithm. Middle: Modified Gram-Schmidt algorithm. Right: Alternative Modified Gram-Schmidt algorithm. In this last algorithm, every time a new column, q_1, of Q is computed, each column of A_2 is updated so that its component in the direction of q_1 is is subtracted out. This means that at the start and finish of the current iteration, the columns of A_L are mutually orthonormal and the columns of A_R are orthogonal to the columns of A_L.

Next, we massage the MGS algorithm into the alternative MGS algorithmic variant given in Figure 3.2.4.2 (right).



The video discusses how MGS can be rearranged so that every time a new vector q_k is computed (overwriting a_k), the remaining vectors, $\{a_{k+1}, \ldots, a_{n-1}\}$, can be updated by subtracting out the component in the direction of q_k. This is also illustrated through the next sequence of equivalent algorithms.

for $j = 0, \ldots, n-1$
  $\rho_{j,j} := \|a_j\|_2$
  $a_j := a_j / \rho_{j,j}$
  for $k = j+1, \ldots, n-1$
    $\rho_{j,k} := a_j^H a_k$
    $a_k := a_k - \rho_{j,k} a_j$
  end
end

(c) MGS algorithm that normalizes the jth column to have unit length to compute q_j (overwriting a_j with the result) and then subtracts the component in the direction of q_j off the rest of the columns $(a_{j+1}, \ldots, a_{n-1})$.

for $j = 0, \ldots, n-1$
  $\rho_{j,j} := \|a_j\|_2$
  $a_j := a_j / \rho_{j,j}$
  for $k = j+1, \ldots, n-1$
    $\rho_{j,k} := a_j^H a_k$
  end
  for $k = j+1, \ldots, n-1$
    $a_k := a_k - \rho_{j,k} a_j$
  end
end

(d) Slight modification of the algorithm in (c) that computes $\rho_{j,k}$ in a separate loop.

for $j = 0, \ldots, n-1$

$\rho_{j,j} := \|a_j\|_2$

$a_j := a_j / \rho_{j,j}$

$\left(\begin{array}{c|c|c} \rho_{j,j+1} & \cdots & \rho_{j,n-1} \end{array} \right) :=$

$a_j^H \left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)$

$\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right) :=$

$\left(\begin{array}{c|c|c} a_{j+1} - \rho_{j,j+1} a_j & \cdots & a_{n-1} - \rho_{j,n-1} a_j \end{array} \right)$

end

(e) Algorithm in (d) rewritten to expose only the outer loop.

for $j = 0, \ldots, n-1$

$\rho_{j,j} := \|a_j\|_2$

$a_j := a_j / \rho_{j,j}$

$\left(\begin{array}{c|c|c} \rho_{j,j+1} & \cdots & \rho_{j,n-1} \end{array} \right) :=$

$a_j^H \left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)$

$\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right) :=$

$\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)$

$- a_j \left(\begin{array}{c|c|c} \rho_{j,j+1} & \cdots & \rho_{j,n-1} \end{array} \right)$

end

(f) Algorithm in (e) rewritten to expose the row-vector-times matrix multiplication $a_j^H \left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)$ and rank-1 update $\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right) - a_j \left(\begin{array}{c|c|c} \rho_{j,j+1} & \cdots & \rho_{j,n-1} \end{array} \right)$.

Figure 3.2.4.3. Various equivalent MGS algorithms.

This discussion shows that the updating of future columns by subtracting out the component in the direction of the latest column of Q to be computed can be cast in terms of a rank-1 update. This is also captured, using FLAME notation, in the algorithm in Figure 3.2.4.2, as is further illustrated in Figure 3.2.4.4:

Algorithm: $[A,R] := \text{MGS}(A)$

Partition $A \rightarrow \left(\begin{array}{c|c} A_L & A_R \end{array} \right)$,

$R \rightarrow \left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right)$

where A_L has 0 columns and R_{TL} is 0×0

while $n(A_L) < n(A)$ **do**

Repartition

$$\left(\begin{array}{c|c} A_L & A_R \end{array} \right) \rightarrow \left(\begin{array}{c|c|c} A_0 & a_1 & A_2 \end{array} \right),$$

$$\left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c|c|c} R_{00} & r_{01} & R_{02} \\ \hline 0 & \rho_{11} & r_{12}^T \\ \hline 0 & 0 & R_{22} \end{array} \right)$$

$\rho_{11} := \|a_1\|_2$

$a_1 := a_1 / \rho_{11}$

$r_{12}^T := a_1^H A_2$

$A_2 := A_2 - a_1 r_{12}^T$

Continue with

$$\left(\begin{array}{c|c} A_L & A_R \end{array} \right) \leftarrow \left(\begin{array}{c|c|c} A_0 & a_1 & A_2 \end{array} \right),$$

$$\left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right) \leftarrow \left(\begin{array}{c|c|c} R_{00} & r_{01} & R_{02} \\ \hline 0 & \rho_{11} & r_{12}^T \\ \hline 0 & 0 & R_{22} \end{array} \right)$$

endwhile

for $j = 0, \ldots, n-1$

$\rho_{j,j} := \|a_j\|_2 \qquad (\rho_{11} := \|a_1^\perp\|_2)$

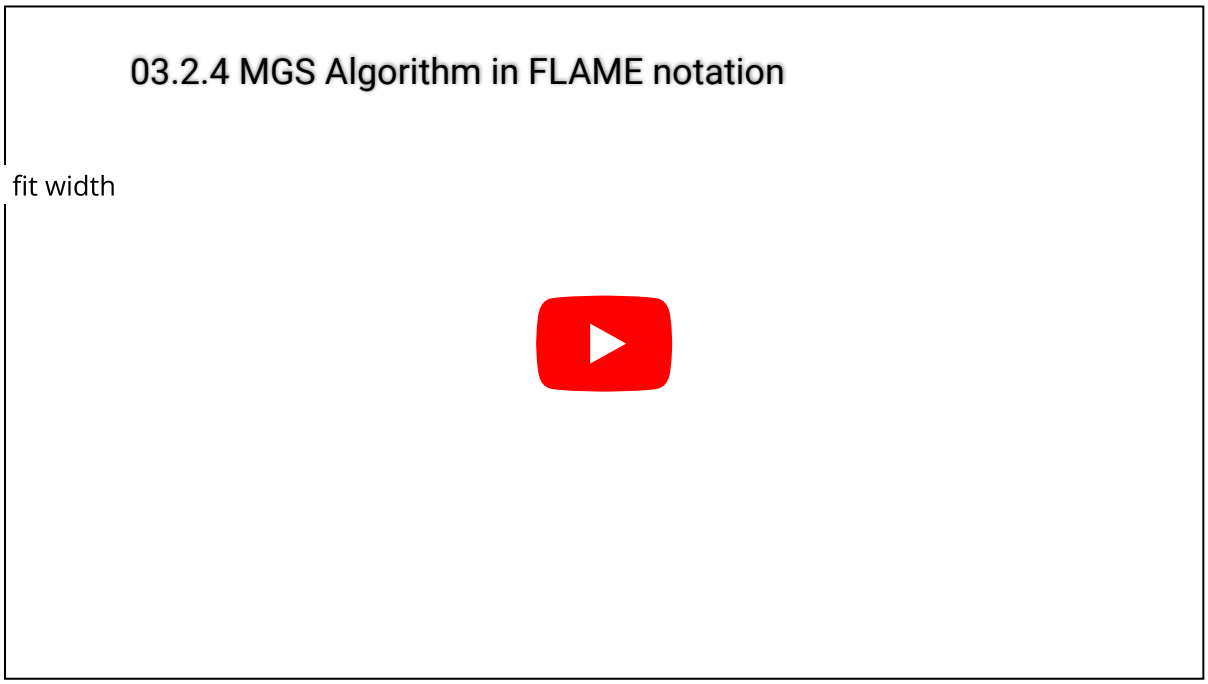
$a_j := a_j / \rho_{j,j} \qquad (a_1 := a_1 / \rho_{11})$

$$\overbrace{\left(\begin{array}{c|c|c} \rho_{j,j+1} & \cdots & \rho_{j,n-1} \end{array} \right)}^{r_{12}^T} := \underbrace{a_j^H}_{a_1^H} \underbrace{\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)}_{A_2}$$

$$\overbrace{\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)}^{A_2} := \overbrace{\left(\begin{array}{c|c|c} a_{j+1} & \cdots & a_{n-1} \end{array} \right)}^{A_2} - \underbrace{a_j}_{a_1} \underbrace{\left(\begin{array}{c|c|c} \rho_{j,j+1} & \cdots & \rho_{j,n-1} \end{array} \right)}_{r_{12}^T}$$

end

Figure 3.2.4.4. Alternative Modified Gram-Schmidt algorithm for computing the QR factorization of a matrix A.



Ponder This 3.2.4.2. Let A have linearly independent columns and let $A = QR$ be a QR factorization of A. Partition

$$A \rightarrow \left(\begin{array}{c|c} A_L & A_R \end{array} \right), \quad Q \rightarrow \left(\begin{array}{c|c} Q_L & Q_R \end{array} \right), \quad \text{and} \quad R \rightarrow \left(\begin{array}{c|c} R_{TL} & R_{TR} \\ \hline 0 & R_{BR} \end{array} \right),$$

where A_L and Q_L have k columns and R_{TL} is $k \times k$.

As you prove the following insights, relate each to the algorithm in Figure 3.2.4.4. In particular, at the top of the loop of a typical iteration, how have the different parts of A and R been updated?

1. $A_L = Q_L R_{TL}$.

   ($Q_L R_{TL}$ equals the QR factorization of A_L.)

2. $\mathcal{C}(A_L) = \mathcal{C}(Q_L)$.

   (The first k columns of Q form an orthonormal basis for the space spanned by the first k columns of A.)

3. $R_{TR} = Q_L^H A_R$.

4. $(A_R - Q_L R_{TR})^H Q_L = 0$.

   (Each column in $A_R - Q_L R_{TR}$ equals the component of the corresponding column of A_R that is orthogonal to $\mathrm{Span}(Q_L)$.)

5. $\mathcal{C}(A_R - Q_L R_{TR}) = \mathcal{C}(Q_R)$.

6. $A_R - Q_L R_{TR} = Q_R R_{BR}$.

   (The columns of Q_R form an orthonormal basis for the column space of $A_R - Q_L R_{TR}$.)

▶ Solution

Homework 3.2.4.3. Implement the algorithm in Figure 3.2.4.4 as

`function [ Aout, Rout ] = MGS_QR( A, R )`

Input is an $m \times n$ matrix A and a $n \times n$ matrix R. Output is the matrix Q, which has overwritten matrix A, and the upper triangular matrix R. (The values below the diagonal can be arbitrary.) You may want to use `Assignments/Week03/matlab/test_MGS_QR.m` to check your implementation.

▶ Solution